

Mérési jegyzőkönyv

OpenCL beadandó – MergeSort és QuickSort
algoritmusok vizsgálata

Készítette: Bodó Tamás HDK6NX
Tantárgy: Párhuzamos eszközök programozása

Tartalomjegyzék

1. A mérés célja.....	3
2. Mérési környezet és módszertan.....	3
3. A párhuzamosítás módja	3
4. A mérési tartomány értékelése	4
5. A mérési eredmények statisztikai feldolgozása.....	4
6. Mérési eredmények és ábrák értékelése.....	6
7. Összegzés és következtetés	11

1. A mérés célja

A mérés célja a MergeSort és a QuickSort rendezőalgoritmusok CPU-s és OpenCL-es megvalósításának összehasonlítása volt. A vizsgálat során azt elemeztem, hogy az egyes algoritmusok hogyan teljesítenek különböző elemszámok mellett, valamint, hogy az OpenCL használata jelent-e gyorsulást kis méretű adathalmazok esetén.

A mérések során három bemeneti méretet vizsgáltam: 1000, 2000 és 3000 elemet. Minden elemszámhoz 30 mérési minta tartozott. A mérési eredményekből átlagot, minimumot, maximumot, mediánt, szórást és relatív szórást is számítottam, hogy ne csak az átlagos futási idő, hanem a mérések eloszlása is értékelhető legyen.

2. Mérési környezet és módszertan

A mérések Apple M1 eszközön készültek. A CPU-s változatoknál a rendezés teljes futási idejét mértem. Az OpenCL-es változatoknál kétféle időt különítettem el:

- OpenCL kernelidő: csak a GPU-n futó kernel tényleges végrehajtási ideje.
- OpenCL teljes idő: az OpenCL-es megoldás teljes futási ideje, amely tartalmazza az adatmozgatást, a memóriakezelést, a kernel indítását, a szinkronizációt és az eredmények visszaolvasását is.

A két OpenCL-es idő elkülönítése fontos, mert a kernelidő önmagában csak a számítási rész sebességét mutatja, míg a teljes OpenCL idő a program szempontjából ténylegesen érzékelhető futási időt adja meg.

3. A párhuzamosítás módja

A MergeSort OpenCL-es változata a rendezési folyamatot több részfeladatra bontja. A MergeSort algoritmus szerkezete alkalmasabb párhuzamos feldolgozásra, mert az összefésülési lépésekben több, egymástól független résztartomány kezelhető. Az OpenCL-es megoldásban a host program indítja a kerneleket, a kernel pedig az adott részfeladatokon dolgozik. A rendezés több lépésből áll, és a lépések között szinkronizációra van szükség.

A QuickSort OpenCL-es változatánál fontos megjegyezni, hogy a jelenlegi megvalósítás nem használja ki teljes mértékben a GPU tömeges párhuzamosságát. A kernelben csak egy munkaszál végzi a rendezést, a többi munkaszál kilép. Emiatt ez inkább egy GPU-n futtatott szekvenciális QuickSort megoldásnak tekinthető, nem pedig teljes értékű párhuzamos QuickSortnak. A rendezés veremalapú, iteratív megoldással történik, amely a rekurzív

QuickSort

működését

helyettesíti.

Ez a különbség a mérési eredmények értelmezésénél is fontos. A MergeSort szerkezete jobban illeszkedik a párhuzamos feldolgozáshoz, míg a QuickSort jelenlegi OpenCL-es megoldása inkább azt mutatja meg, hogy kis bemeneti méretek esetén a GPU-ra történő áthelyezés önmagában nem jelent automatikus gyorsulást.

4. A mérési tartomány értékelése

A mérések 1000, 2000 és 3000 elemre készültek. Ez a tartomány viszonylag szűk, és csak három mérési pontot tartalmaz, ezért az algoritmusok hosszabb távú skálázódása ebből csak korlátozottan ítéltető meg. Nagyobb elemszámokkal és több mérési ponttal pontosabban látható lenne, hogy a futási idők hogyan követik az elméletben várható növekedési jelleget.

A szekvenciális CPU-s mérések esetén az elméleti várakozás szerint mind a MergeSort, mind a QuickSort futási ideje átlagos esetben $n \log n$ jellegű növekedést mutat. A jelenlegi kis elemszámtartományban ez a jelleg csak részben figyelhető meg. Ennek oka, hogy a futási idők nagyon kicsik, ezredmásodperces nagyságrendűek, ezért a rendszerterhelés, az operációs rendszer ütemezése, a cache állapota és egyéb mérési zajok arányaiban jelentősen befolyásolhatják az eredményeket.

A mérési eredményekből ezért nem célszerű általános következtetést levonni nagy bemenetekre. A jelenlegi mérés elsősorban a kis elemszámokra vonatkozó viselkedést mutatja meg. Ebben a tartományban jól megfigyelhető, hogy az OpenCL teljes futási idejét jelentősen befolyásolják az indítási, adatmozgatási és szinkronizációs többletköltségek.

5. A mérési eredmények statisztikai feldolgozása

Az átlagos futási idő önmagában nem írja le teljesen a mérések viselkedését, ezért a 30 mérési minta alapján meghatároztam a mediánt, a minimumot, a maximumot, a mintaszórást és a relatív szórást is.

A medián azért hasznos, mert kevésbé érzékeny a kiugró értékekre, mint az átlag. A szórás azt mutatja meg, hogy az egyes mérések mennyire térnek el az átlagtól. A relatív szórás százalékos formában adja meg a szórás és az átlag arányát.

Relatív szórás képlete:

$$\text{relatív szórás} = \text{szórás} / \text{átlag} * 100$$

Statisztikai összesítés a 30 mérési minta alapján:

Algoritmus / mérés	Elemszám	Átlag (ms)	Medián (ms)	Minimum (ms)	Maximum (ms)	Szórás (ms)	Relatív szórás
MergeSort CPU	1000	0.069911	0.069792	0.068458	0.074750	0.001097	1.57%

MergeSort CPU	2000	0.149220	0.148938	0.148250	0.152542	0.000969	0.65%
MergeSort CPU	3000	0.216549	0.200542	0.198417	0.245333	0.018472	8.53%
MergeSort OpenCL teljes	1000	4.994168	4.940396	4.111292	7.607417	0.610201	12.22%
MergeSort OpenCL teljes	2000	6.356704	6.369479	5.848375	6.808458	0.235354	3.70%
MergeSort OpenCL teljes	3000	8.812182	8.823542	8.132167	9.537750	0.386284	4.38%
MergeSort OpenCL kernel	1000	0.028899	0.026205	0.025277	0.076395	0.009450	32.70%
MergeSort OpenCL kernel	2000	0.050733	0.050202	0.049518	0.055773	0.001447	2.85%
MergeSort OpenCL kernel	3000	0.092625	0.090281	0.089660	0.104009	0.004104	4.43%

Algoritmus / mérés	Elemszám	Átlag (ms)	Medián (ms)	Minimum (ms)	Maximum (ms)	Szórás (ms)	Relatív szórás
QuickSort CPU	1000	0.121708	0.120334	0.117417	0.137458	0.004157	3.42%
QuickSort CPU	2000	0.231571	0.215708	0.203292	0.262292	0.023233	10.03%
QuickSort CPU	3000	0.315850	0.321688	0.271833	0.350417	0.020277	6.42%
QuickSort OpenCL teljes	1000	4.291800	3.882313	2.867667	13.237208	1.783022	41.54%
QuickSort OpenCL teljes	2000	4.657253	4.306063	3.925333	7.416292	0.878804	18.87%
QuickSort OpenCL teljes	3000	6.432017	6.395375	6.113875	6.833042	0.199591	3.10%
QuickSort OpenCL kernel	1000	0.077493	0.071512	0.050212	0.125171	0.017138	22.11%
QuickSort OpenCL kernel	2000	0.092685	0.085917	0.077457	0.154392	0.019386	20.92%
QuickSort OpenCL kernel	3000	0.129493	0.128931	0.122574	0.136264	0.003588	2.77%

A táblázat alapján megfigyelhető, hogy a CPU-s MergeSort 1000 és 2000 elem esetén stabil eredményeket adott, mert a relatív szórás alacsony volt. 3000 elemnél viszont nagyobb ingadozás jelent meg, amit az átlag és a medián eltérése is mutat. Az átlag 0.216549 ms, míg a medián 0.200542 ms, tehát néhány nagyobb érték felfelé húzta az átlagot.

A QuickSort CPU-s méréseinél szintén látható ingadozás, főleg 2000 elemnél, ahol a relatív szórás 10.03%. Ez magyarázza, hogy a szekvenciális mérési görbe nem teljesen az elméletben

várható, egyenletes $n \log n$ jellegű növekedést mutatja. A futási idők nagyon kicsik, ezért a mérési zaj és a rendszer pillanatnyi állapota arányaiban jelentősen befolyásolhatta az eredményeket.

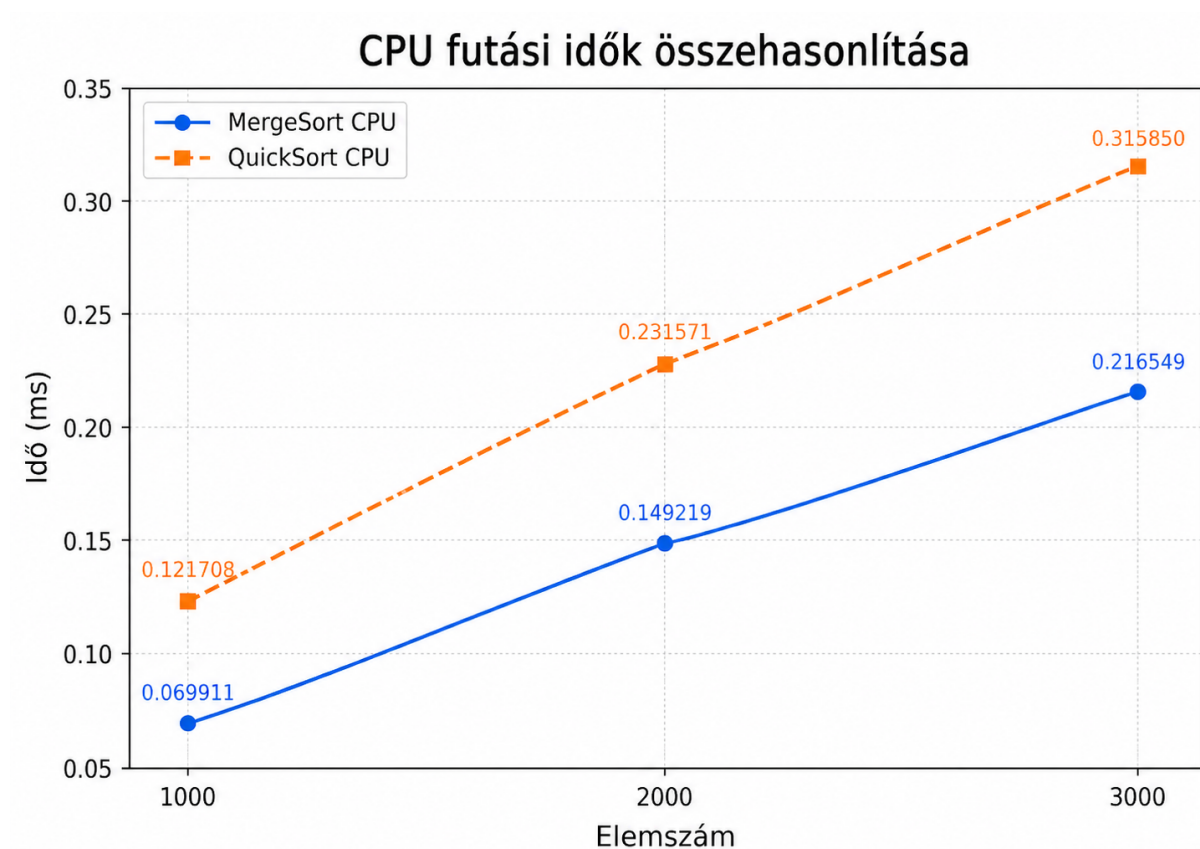
Az OpenCL teljes futási idők esetén a QuickSort 1000 és 2000 elemnél nagyobb relatív szórást mutatott. Ennek oka főként néhány kiugró mérési érték, például 1000 elemnél a 13.237208 ms-os maximum. Ez arra utal, hogy az OpenCL teljes futási idejét nemcsak a számítási rész, hanem az inicializáció, az adatmozgatás, a kernelindítás és a szinkronizáció is jelentősen befolyásolja.

A kernelidők általában kisebbek, mint a teljes OpenCL futási idők, de itt is előfordulnak kiugró értékek. Ez különösen a nagyon rövid, ezredmásodperces futásoknál látványos, ahol egyetlen lassabb mérés is nagy relatív szórást eredményezhet.

6. Mérési eredmények és ábrák értékelése

A következő ábrák a CPU-s és OpenCL-es futási időket mutatják be. A logaritmikus skála használata több ábrán is indokolt, mert a CPU-idők, az OpenCL kernelidők és az OpenCL teljes futási idők eltérő nagyságrendbe esnek. Logaritmikus skálán ezek az értékek egy ábrán is áttekinthetőbben összehasonlíthatók.

1. ábra: CPU futási idők összehasonlítása

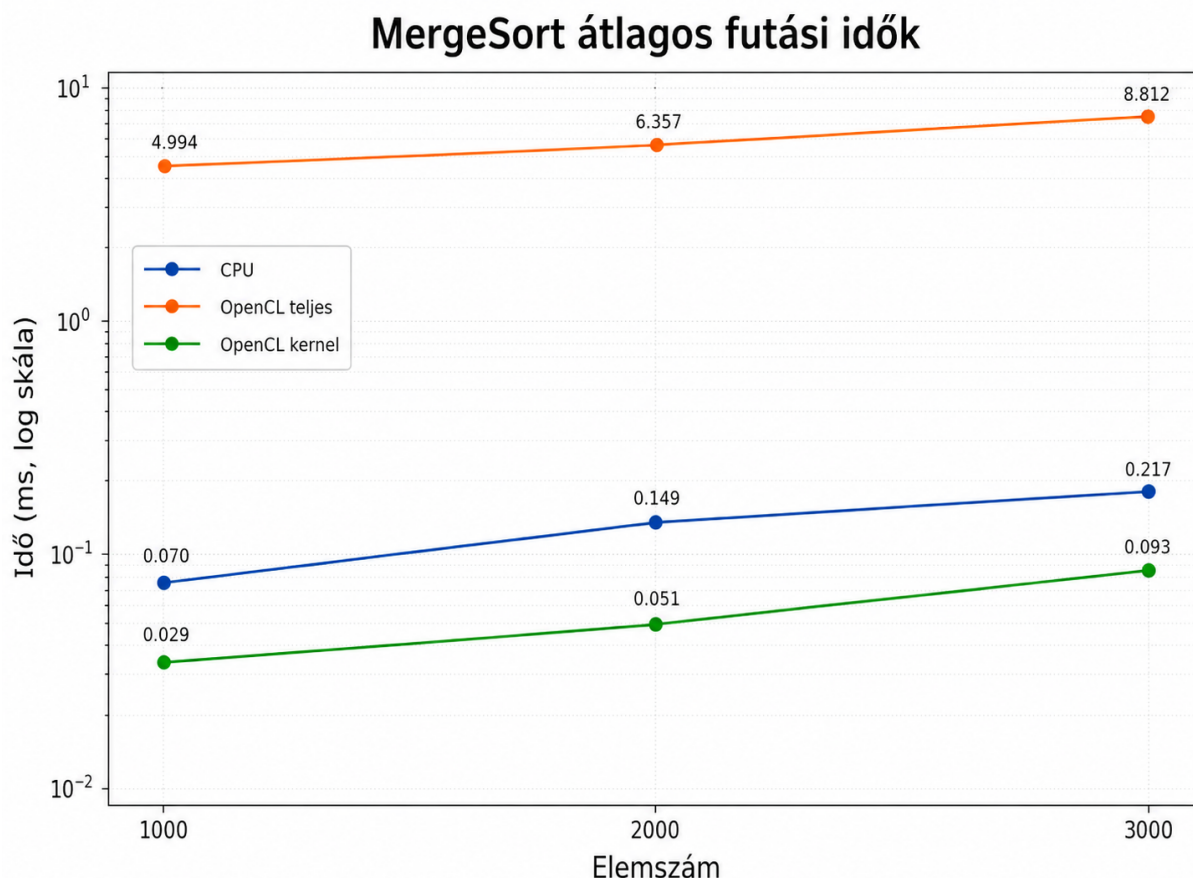


Az 1. ábra a MergeSort és QuickSort CPU-s futási idejét hasonlítja össze 1000, 2000 és 3000 elem esetén. Látható, hogy mindkét algoritmus futási ideje növekszik az elemszám növekedésével. A mérések alapján CPU-n a MergeSort minden vizsgált elemszámnál gyorsabb volt, mint a QuickSort. Ez abból látszik, hogy a MergeSort értékei végig alacsonyabban helyezkednek el a diagramon.

1000 elem esetén a MergeSort átlagos CPU ideje 0.069911 ms, míg a QuickSort CPU ideje 0.121708 ms volt. 3000 elemnél a MergeSort 0.216549 ms, a QuickSort pedig 0.315850 ms alatt futott le.

A CPU-s méréseknél fontos megjegyezni, hogy a három mérési pont és a kis elemszámtartomány miatt a görbék nem mutatják teljesen tisztán az elméletben várható $n \log n$ növekedési jelleget. A statisztikai összesítés alapján több CPU-s mérési sorozatnál is látható ingadozás, ezért az ábra elsősorban a kis bemeneti méretekre vonatkozó tapasztalati összehasonlításként értelmezhető.

2. ábra: MergeSort átlagos futási idők

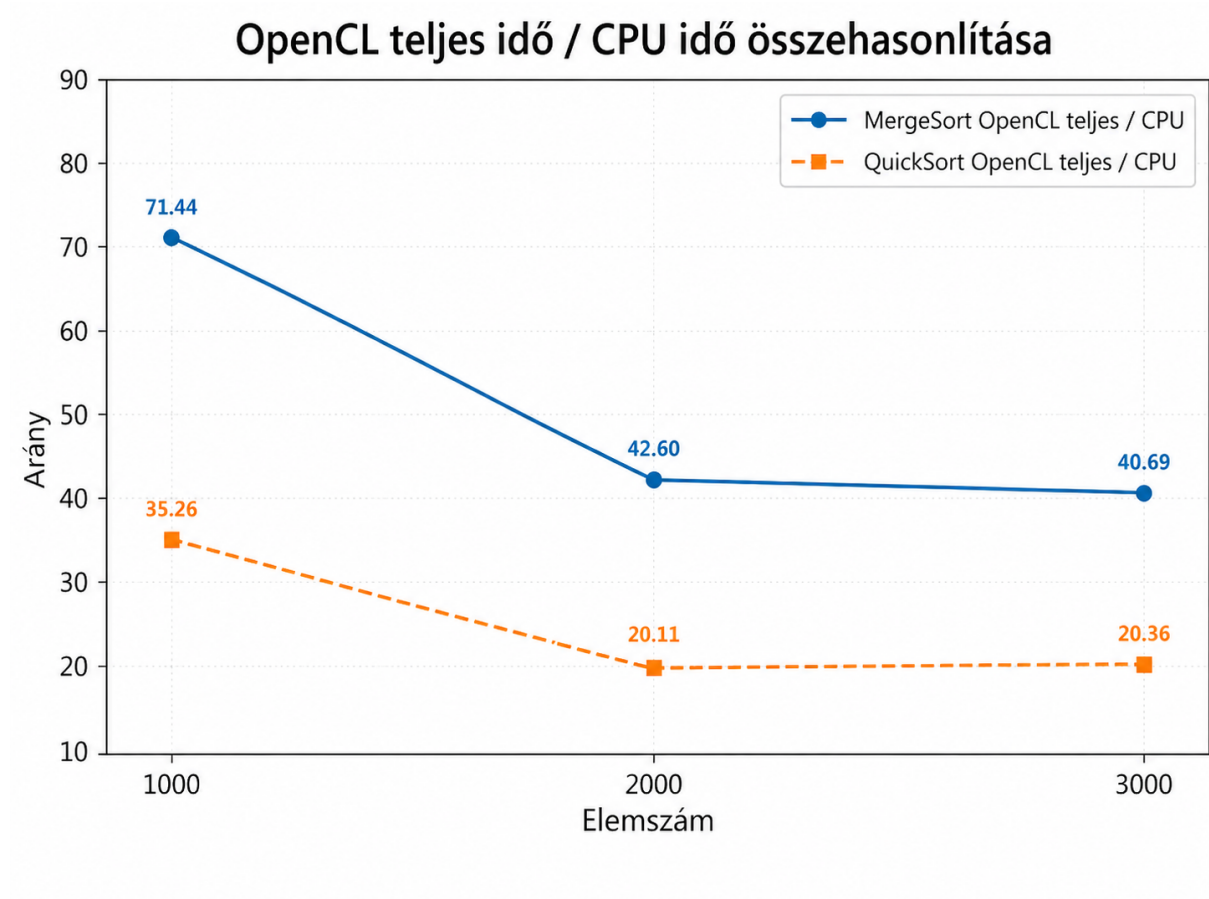


A 2. ábra a MergeSort CPU-s, OpenCL teljes és OpenCL kernel futási idejét mutatja. A diagram logaritmikus skálát használ, mert a teljes OpenCL idő jóval nagyobb, mint a CPU és a kernelidő. A logaritmikus skála használata indokolt, mert így az eltérő nagyságrendű értékek egy ábrán is jól összehasonlíthatók.

Megfigyelhető, hogy maga az OpenCL kernel nagyon gyorsan lefut, viszont a teljes OpenCL futási idő sokkal nagyobb. 1000 elemnél a MergeSort kernelideje 0.028899 ms volt, míg a teljes OpenCL idő 4.994168 ms. 3000 elemnél a kernelidő 0.092625 ms, a teljes OpenCL idő pedig 8.812182 ms.

Ez azt jelenti, hogy a futási idő jelentős részét nem a számítás, hanem az OpenCL környezet használata, az adatmozgatás, a kernelindítás és a szinkronizáció adja.

3. ábra: OpenCL teljes idő / CPU idő összehasonlítása

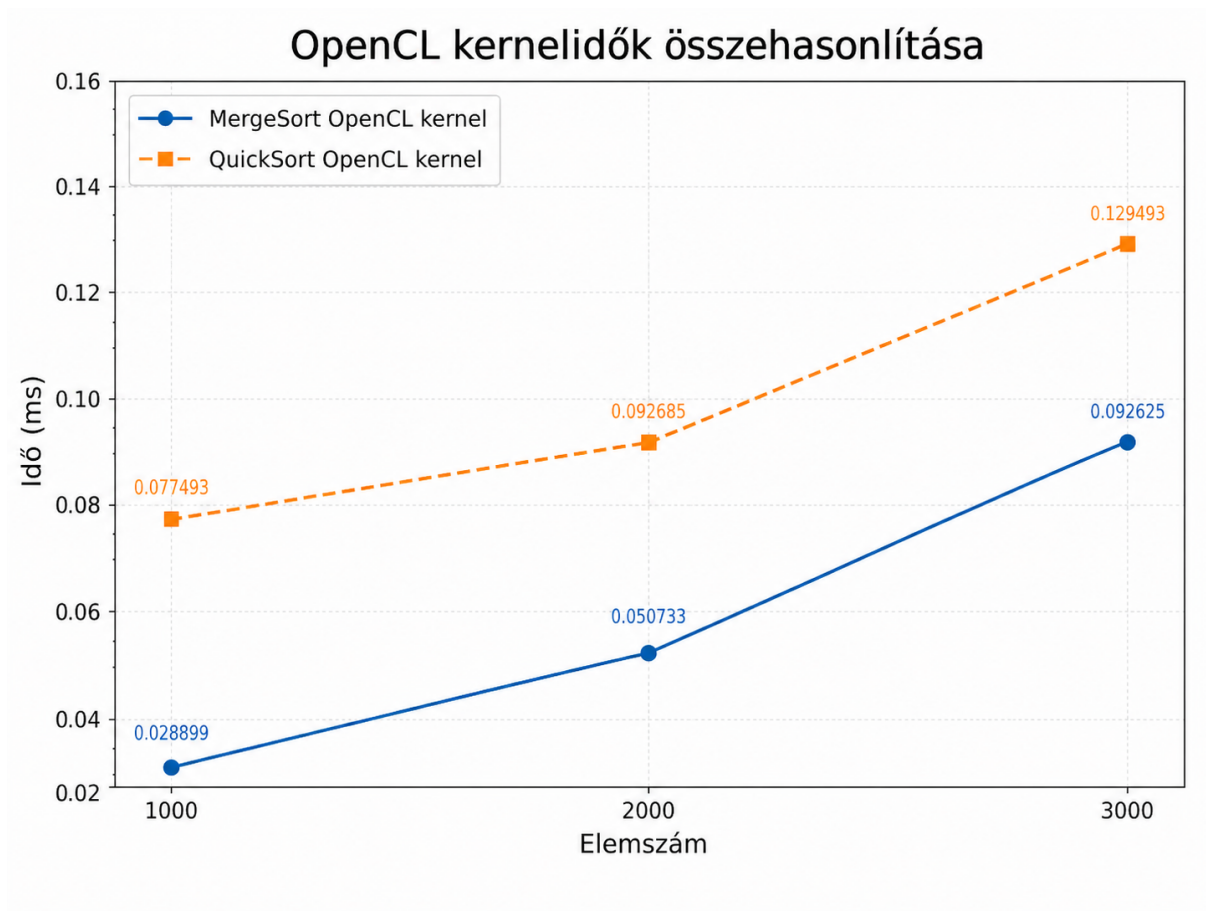


A 3. ábra azt mutatja meg, hogy az OpenCL teljes futási ideje hány-szorosa a CPU-s futási időnek. Mindkét algoritmusnál látható, hogy a vizsgált kis elemszámok mellett az OpenCL teljes futási ideje jelentősen nagyobb, mint a CPU-s futási idő.

MergeSort esetén ez az arány 1000 elemnél körülbelül 71-szeres, 2000 elemnél körülbelül 43-szoros, 3000 elemnél pedig körülbelül 41-szeres volt. QuickSort esetén az arány kisebb, de még így is jelentős: 1000 elemnél körülbelül 35-szörös, 2000 és 3000 elemnél pedig körülbelül 20-szoros.

Az arányok csökkenése azt jelzi, hogy az elemszám növelésével az OpenCL többletköltségének relatív hatása mérséklődik. Ugyanakkor a vizsgált tartományban még mindig a CPU-s megoldás volt gyorsabb teljes futási idő alapján.

4. ábra: OpenCL kernelidők összehasonlítása

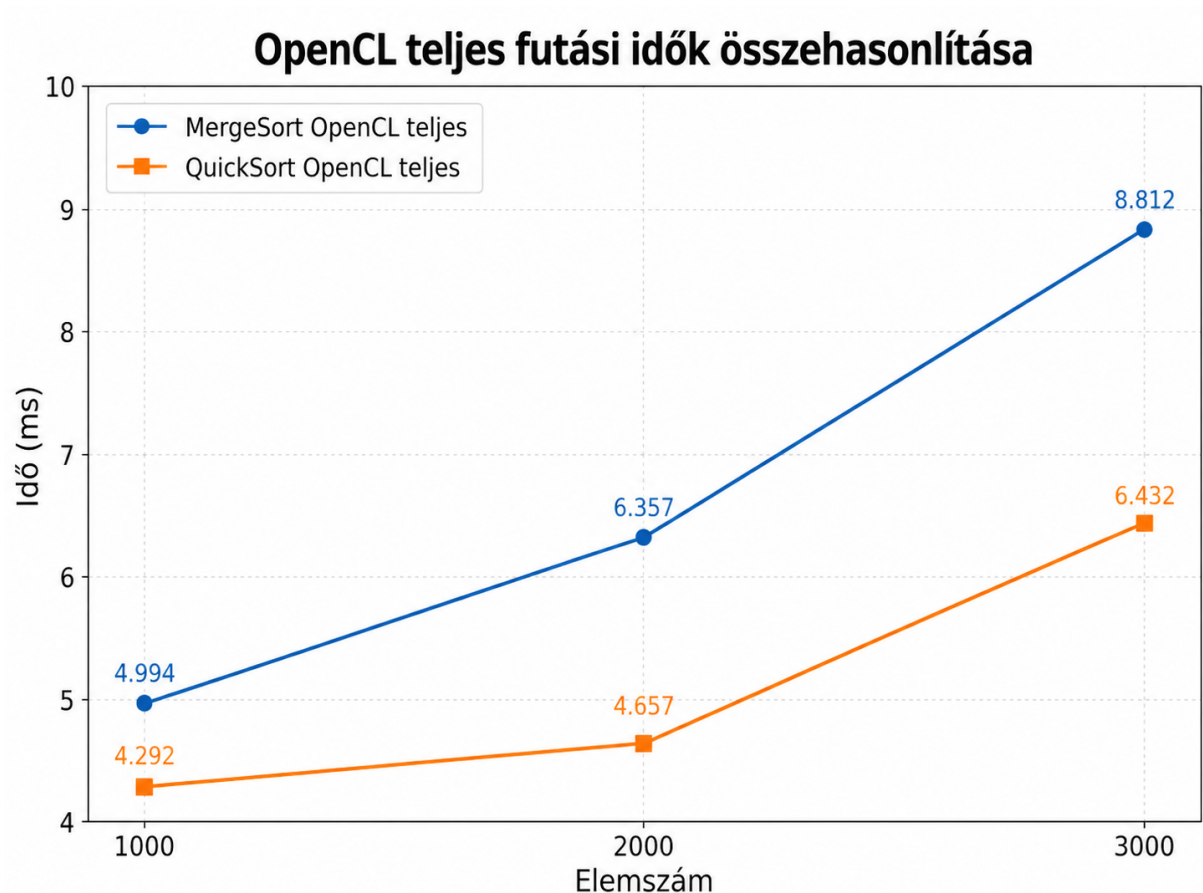


A 4. ábra a MergeSort és QuickSort OpenCL kernelidejét hasonlítja össze. A kernelidő csak a tényleges GPU-n futó számítás idejét mutatja, nem tartalmazza az adatmásolást, a memóriakezelést és az OpenCL indítási költségeit.

A diagram alapján a kernelidők nagyon alacsonyak, ezredmásodperces nagyságrendűek. A MergeSort kernelideje minden elemszámnál kisebb volt, mint a QuickSort kernelideje. 1000 elemnél a MergeSort kernelideje 0.028899 ms, míg a QuickSort kernelideje 0.077493 ms volt. 3000 elemnél a MergeSort kernelideje 0.092625 ms, a QuickSort kernelideje pedig 0.129493 ms volt.

Ez alapján a MergeSort OpenCL kernel szintjén kedvezőbb eredményt adott. Ennek oka lehet, hogy a MergeSort szerkezete jobban illeszkedik az OpenCL-es, több munkadarabra bontható feldolgozáshoz, míg a QuickSort jelenlegi OpenCL megvalósítása csak korlátozottan használja ki a párhuzamosságot.

5. ábra: OpenCL teljes futási idők összehasonlítása

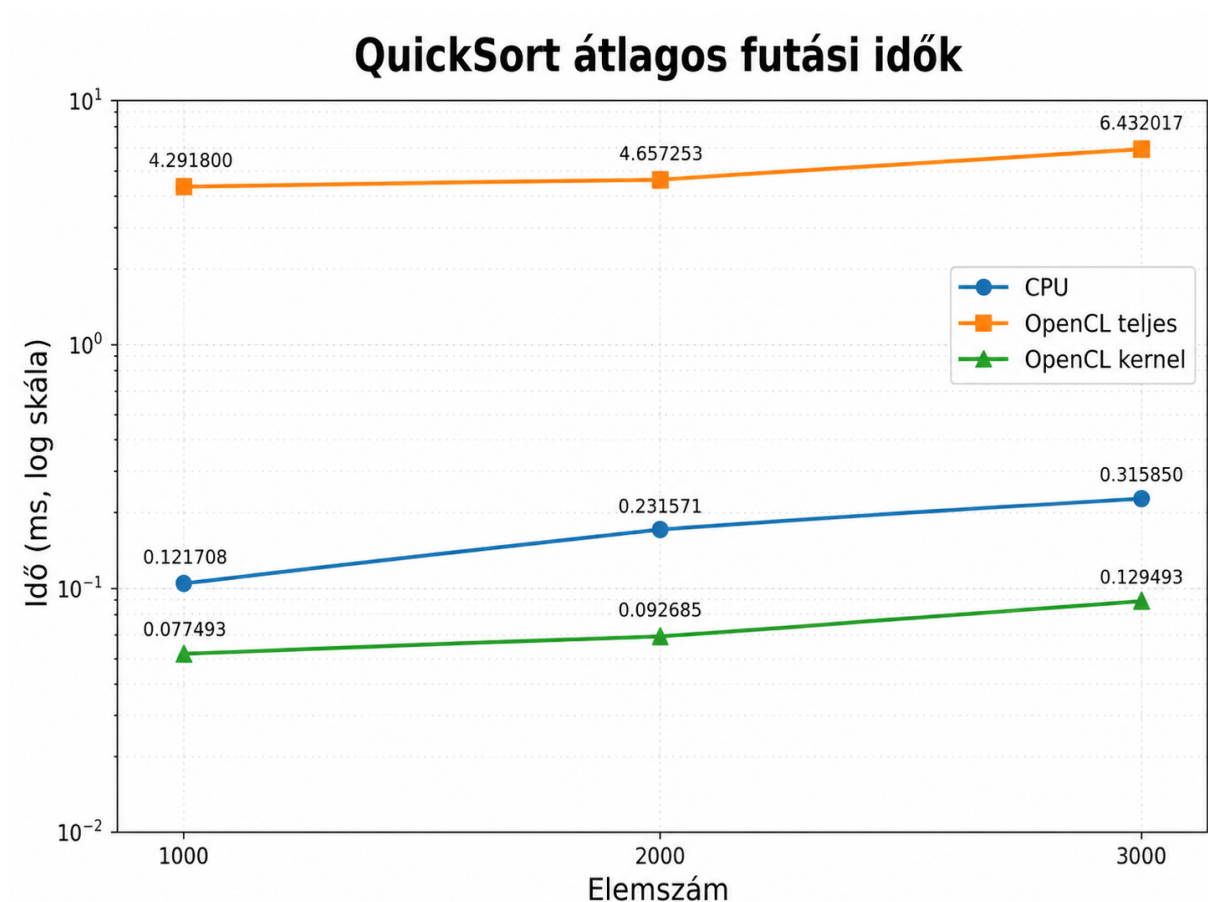


Az 5. ábra a MergeSort és QuickSort teljes OpenCL futási idejét hasonlítja össze. A teljes OpenCL idő tartalmazza a kernel futását, az adatmozgatást, a memóriakezelést, az OpenCL műveletek indítási többletköltségét és a szinkronizációt is.

A diagram alapján a QuickSort teljes OpenCL futási ideje alacsonyabb volt, mint a MergeSort teljes OpenCL futási ideje. Ez főleg 3000 elemnél látszik erősen, ahol a MergeSort OpenCL teljes ideje 8.812182 ms, míg a QuickSort esetén 6.432017 ms volt.

Ez érdekes eredmény, mert kernelidő alapján a MergeSort volt gyorsabb, teljes OpenCL idő alapján viszont a QuickSort adott alacsonyabb értékeket. Ez azt mutatja, hogy a teljes futási időt nem kizárólag a kernel sebessége határozza meg, hanem az OpenCL környezet használatával járó egyéb költségek is jelentős szerepet játszanak.

6. ábra: QuickSort átlagos futási idők



A 6. ábra a QuickSort CPU-s, OpenCL teljes és OpenCL kernel futási idejét mutatja. Itt is logaritmusos skála szerepel, hogy a CPU, az OpenCL teljes idő és a kernelidő egyszerre jól látható legyen.

A mérések alapján a QuickSort kernelideje kicsi, viszont a teljes OpenCL futási idő jóval nagyobb, mint a CPU-s futási idő. 1000 elemnél a QuickSort CPU-s ideje 0.121708 ms, az OpenCL kernelideje 0.077493 ms, a teljes OpenCL futási idő pedig 4.291800 ms volt. 3000 elemnél a CPU-s idő 0.315850 ms, a kernelidő 0.129493 ms, a teljes OpenCL idő pedig 6.432017 ms volt.

Ez megerősíti, hogy kis elemszámoknál az OpenCL többletköltsége nagyobb hatással van a teljes futásra, mint maga a GPU-n futó számítás előnye. A QuickSort esetében különösen fontos, hogy a jelenlegi kernel nem teljesen párhuzamosított, ezért az OpenCL-es futtatás nem tudja kihasználni a GPU teljes számítási kapacitását.

7. Összegzés és következtetés

A mérések alapján megállapítható, hogy a vizsgált 1000, 2000 és 3000 elemszámok mellett a CPU-s megoldások gyorsabbak voltak, mint az OpenCL-es megoldások teljes futási ideje.

Ennek oka, hogy ilyen kis adatmennyiségnél az OpenCL használatával járó többletköltségek – például az adatmásolás, a memóriakezelés, a kernelindítás és a szinkronizáció – nagyobbak, mint amennyi időt a párhuzamos végrehajtás meg tudna takarítani.

Fontos különbség látható a teljes OpenCL idő és a kernelidő között. A kernelidők nagyon alacsonyok, tehát maga a GPU-n futó számítás gyors. A teljes OpenCL futási idő viszont jóval nagyobb, ezért a teljes program szempontjából a gyorsulás ezeknél a kis elemszámoknál nem jelentkezik.

CPU-n a MergeSort bizonyult gyorsabbnak, míg az OpenCL teljes futási időket nézve a QuickSort adott kedvezőbb eredményeket. Kernelidő alapján ugyanakkor a MergeSort volt gyorsabb. Ez azt mutatja, hogy az algoritmusok összehasonlításánál külön kell kezelni a tényleges számítási időt és a teljes futási időt.

A mérés korlátja, hogy csak három elemszámmra készült, ezért a tartomány és a felbontás nem elegendő az algoritmusok teljes skálázódásának pontos vizsgálatához. A statisztikai feldolgozás alapján látható, hogy egyes mérési sorozatoknál a szórás és a relatív szórás nem elhanyagolható, ezért az átlagértékek mellett az eloszlás jellemzőit is figyelembe kell venni. Ez különösen a szekvenciális CPU-s méréseknél és az OpenCL teljes futási időknél fontos, ahol néhány kiugró érték érezhetően módosította az átlagot.

Összességében a mérés azt mutatja, hogy az OpenCL előnye nem automatikusan jelentkezik. Kis adathalmazoknál a többletköltség dominál, ezért a CPU-s rendezés kedvezőbb lehet. Az OpenCL várhatóan nagyobb elemszámoknál lenne előnyösebb, ahol a párhuzamosításból származó nyereség már ellensúlyozhatja az adatmozgatási, kernelindítási és szinkronizációs költségeket.